

CI/CD

Arturo Silvelo

Try New Roads

Ejercicio: Configurar hooks locales en un repositorio Python

En este ejercicio, vamos a configurar dos hooks locales en un repositorio de Python para automatizar tareas comunes:

1. **Hook** `pre-commit`: Validará los archivos Python staged utilizando un linter (`flake8`) antes de permitir un commit.
2. **Hook** `commit-msg`: Ejecutará los tests del proyecto utilizando `pytest` antes de permitir un push.

Pasos a seguir:

1. Preparar el entorno:

- Crea un repositorio nuevo o utiliza uno existente.
- Asegúrate de tener `flake8` y `pytest` instalados en tu entorno:

```
python3 -m venv venv  
source venv/bin/activate  
pip install flake8 black pytest
```

2. Configurar el hook `pre-commit`:

- Crea el archivo `.git/hooks/pre-commit`.
- Escribe un script que valide los archivos Python staged con `flake8`.
- Haz el hook ejecutable:

```
chmod +x .git/hooks/pre-commit
```

```
#!/bin/bash
# filepath: .git/hooks/pre-commit
# Hook pre-commit para ejecutar Flake8 en archivos staged

set -e

# Obtener archivos staged que terminan en .py
FILES=$(git diff --cached --name-only --diff-filter=ACM | grep '\.py$')

if [ -z "$FILES" ]; then
    echo "No Python files to lint."
    exit 0
fi

# Ejecutar Flake8 en los archivos staged
echo "Running Flake8..."
echo "$FILES" | xargs flake8

if [ $? -ne 0 ]; then
    echo "Flake8 found issues. Commit aborted."
    exit 1
fi

echo "Flake8 passed. Proceeding with commit."
exit 0
```

3. Configurar el hook `pre-push`:

- Crea el archivo `.git/hooks/commit-msg`.
- Escribe un script que ejecute los tests del proyecto con `pytest`.
- Haz el hook ejecutable:

```
chmod +x .git/hooks/commit-msg
```

```
#!/bin/bash
# filepath: .git/hooks/pre-push
# Hook pre-push para ejecutar tests con pytest

set -e

echo "Running tests with pytest..."
pytest

if [ $? -ne 0 ]; then
    echo "Tests failed. Push aborted."
    exit 1
fi

echo "All tests passed. Proceeding with push."
exit 0
```

4. Probar los hooks:

- Modifica un archivo Python con errores de estilo y prueba el hook `pre-commit`.
- Intenta hacer un push con tests que fallen y verifica que el hook `commit-msg` lo bloquee.

Pasos a seguir:

1. Preparar el entorno:

- Crea un repositorio nuevo o utiliza uno existente.
- Asegúrate de tener `pre-commit`, `flake8` y `pytest` instalados en tu entorno:

```
python3 -m venv venv  
source venv/bin/activate  
pip install pre-commit flake8 pytest
```

2. Configurar pre-commit:

- Crea un archivo `.pre-commit-config.yaml` en la raíz del repositorio con el siguiente contenido:

Ejercicio: Configurar GitHub Actions para un repositorio Python

Crear un workflow de CI con GitHub Actions que ejecute formateo/lint y tests automáticamente en cada push y pull request.

1. Archivo de workflow en `.github/workflows/ci.yml` que:
 - Se dispare en `push` y `pull_request` sobre ramas `main`.
 - Use `actions/checkout@v4` y `actions/setup-python@v4`.
 - Instale dependencias (`requirements.txt`) y herramientas `black`, `flake8`, `pytest`.
 - Ejecute: `black --check .`, `flake8 .` y `pytest`.

Pasos sugeridos para realizarlo

1. Crear `.github/workflows/ci.yml` con los jobs y steps necesarios.
2. Comprobar en una rama feature: push → abrir PR → verificar ejecución en Actions.
3. Corregir un error intencionado (p. ej. mala formatación o test roto) y comprobar que el workflow falla.

```
repos:
- repo: https://github.com/psf/black
  rev: 23.3.0
  hooks:
    - id: black
      language_version: python3
- repo: https://github.com/pycqa/flake8
  rev: 6.1.0
  hooks:
    - id: flake8
      name: Revisar estilo con flake8
      language_version: python3
- repo: local
  hooks:
    - id: pytest
      name: pytest
      entry: pytest
      language: system
      pass_filenames: false
      stages: [commit-msg]
```

- Este archivo configura dos hooks:
 - `black`, `flake8` para validar los archivos staged en el hook `pre-commit`.
 - `pytest` para ejecutar los tests en el hook `commit-msg`.

3. Instalar los hooks:

- Ejecuta el siguiente comando para instalar los hooks en el repositorio:

```
pre-commit install  
pre-commit install --hook-type commit-msg
```


4. Probar los hooks:

- Modifica un archivo Python con errores de estilo y prueba el hook `pre-`

`commit` :

```
git add <archivo>  
git commit -m "Test pre-commit hook"
```

- Introduce un test que falle y prueba el hook `commit-msg` :

```
git commit
```

```
    name: CI - lint & test

on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main, develop ]
```

```
jobs:
  lint-and-test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        python-version: [3.11]
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Set up Python ${ matrix.python-version }
        uses: actions/setup-python@v4
        with:
          python-version: ${ matrix.python-version }

      - name: Cache pip
        uses: actions/cache@v4
        with:
          path: ~/.cache/pip
          key: pip-${ runner.os }-py${ matrix.python-version }-${ hashFiles('**/requirements.txt') }
          restore-keys: |
            pip-${ runner.os }-py${ matrix.python-version }-
```

- name: Install dependencies
run: |
python -m pip install --upgrade pip
if [-f requirements.txt]; then pip install -r requirements.txt; fi
pip install black flake8 pytest
- name: Run Black (check)
run: |
black --check .
- name: Run Flake8
run: |
flake8 .
- name: Run tests
run: |
pytest -q